

Bob Break

A developer attention management system for agentic AI

While Bob works, you breathe.

1. Problem Statement: The Human Attention Bottleneck

AI development agents can now plan changes, generate code, run tests, read documentation, and coordinate tasks faster than a human developer can comfortably follow. The bottleneck is no longer only development speed; it is the amount of technical information a person can process without becoming overwhelmed.

When IBM Bob runs a multi-step task across several agents, the developer may feel compelled to monitor plans, updates, questions, diffs, and test results continuously. Even while Bob is doing the work, the developer's eyes and brain rarely get a genuine break. This creates visual fatigue, cognitive overload, repeated checking, unnecessary context switching, and interruptions to work that was progressing correctly.

Too much output	Unclear urgency	Anxious waiting
Code, logs, plans and test results compete for attention.	Routine progress and genuine blockers look equally important.	Saved development time becomes continuous supervision.

2. Solution Statement: Bob Break

Bob Break is an attention-management layer for IBM Bob's agentic workflow. It converts agent activity into calm visual progress and interrupts the developer only when human attention creates real value. Routine updates are retained for the final report rather than displayed as a continuous stream of technical text.

BACKGROUND
PROGRESS

Silent aggregation; a plant grows in the garden.

DECISION REQUIRED

The recovery view pauses and presents one short question.

BLOCKER OR RISK

A calm notice or immediate alert explains what needs attention.

COMPLETED

The plant flowers and the result is added to the final summary.

Bob reduces the workload. Bob Break reduces the cognitive load.

3. Demonstrated Workflow: Release Readiness

To prove that Bob Break is more than a wellness widget, the prototype visualizes a concrete developer workflow: preparing a repository for release. Four focused tasks operate in parallel, while the Attention Manager decides what the developer should see.

Task	Technical contribution	Visual representation
Diff analysis	Change surface and categorized commits	Plant 1
Dependency scan	Version drift and breaking-change flags	Plant 2
Test execution	Pass/fail results and failing suites	Plant 3
Documentation review	CHANGELOG draft and stale-doc warnings	Plant 4

4. Technical Execution: IBM Bob 2.0 Utilization

IBM Bob 2.0 is used as the core development partner for planning, building, validating, and documenting the prototype. The submission will preserve screenshots and task summaries as evidence of the exact work Bob performed.

- Agent Mode: reads the project requirements, inspects the repository, and creates the implementation plan.
- Subagents: isolate focused responsibilities for analysis, testing, documentation, and interface work.
- Parallel Tasks: allow independent release-readiness checks to progress concurrently.
- Document Understanding: interprets project requirements, package metadata, README content, accessibility guidance, and release documentation.
- Validation: supports implementation review, unit-test creation, and a structured completion summary.

Integration transparency. The prototype does not claim a live event API from the Bob IDE. Its Event Adapter accepts a deterministic replay of the documented workflow and can also represent events produced by local release-readiness analyzers. A future Bob event integration can replace the adapter without changing the Attention Manager or visual interface.

5. Impact and Measurement

Bob Break instruments the workflow rather than asserting unverified productivity gains. The main measure is the ratio between events produced and events surfaced to the developer. The prototype also records interruptions requiring human input and time from run start to a reviewable report.

Measure	What it demonstrates
Events generated vs. surfaced	How much technical noise the Attention Manager removes
Human decisions required	Whether interruptions are limited to valuable input
Time to reviewable report	How quickly the developer can understand the completed work
Context checks avoided	The reduction in repeated supervision and visual exposure

6. Prototype Experience

The demonstration follows one complete 60-90 second journey. A developer starts the release-readiness run, four plants begin growing as tasks progress, and a breathing animation replaces the continuous log stream. Routine messages remain hidden. When a dependency decision requires human input, the garden pauses and one concise decision card appears. After the answer is submitted, the run continues and the plants flower. The experience ends with a structured release report.

VISUAL RECOVERY	ACTIONABLE INTERRUPTION	CLEAR RETURN
The garden and breathing view reduce exposure to dense technical text.	Only a decision, blocker, or critical risk interrupts the developer.	A concise report explains what changed, passed, failed, and needs review.

Agentic AI manages the work.
Bob Break manages the developer's attention.